

Progress Report 1

María Jaque Oficialdegui, Emma Rey Sánchez,
José Andrés Ridruejo Tuñón, Catalina Royo-Villanova Seguí

April 2024

1 Problem and Hypotheses

This last week, we have decided to base our model in the one described in the “Show and Tell: a neural image caption generator” [1]. Their model was inspired by the then recent encoder-decoder architectures of new machine translation models. What sets it apart from them is that the original Recurrent Neural Network (RNN) encoder is replaced with a pretrained Computer Vision Convolutional Neural Network (CNN). And the decoder is an LSTM.

The training objective of the model is to obtain the parameters that maximize the probability of a correct description S , given an image I . This is the same as maximising the probability of a correct next word, given an image I and the $t - 1$ correct words $S_0, \dots, S_{t-1}$:

$$\theta^* = \arg \max_{\theta} \sum_{t=0}^N \log p(S_t | I, S_0, \dots, S_{t-1}; \theta) \quad (1)$$

Firstly, as stated before, a Convolutional Neural Network has been selected as the encoder. The CNN implemented is based on an Inception version 3 pretrained model. This architecture from the Inception family of CNN introduces several enhancements. These include the application of Label Smoothing, the implementation of factorized 7x7 convolutions, and the deployment of an auxiliary classifier to distribute label information further down the network. Additionally, it utilizes batch normalization in the layers of the side head. However, it is necessary to remove the final softmax layer since instead of classifying the images, we want to extract the main characteristics.

Furthermore, for the decoder part of the model which had to be a RNN, we have decided to implement a Long-Short Term Memory (LSTM) network with one layer. Additionally to this LSTM, there is an embedding layer before the LSTM to standardize their dimensions to match the features and a linear layer through which we pass the output of the LSTM layer to obtain the probability of the next word.

To join both models, the image firstly passes through the CNN to obtain its features. This will be the first input of the LSTM, at time $t - 1$. Then we differentiate between training mode and evaluation mode. In the training mode, the rest of the inputs are the embedding of the caption's next words $E_0, \dots, E_{N-1}$.

$$x_{-1} = \text{CNN}(I) \quad (2)$$

$$x_t = E_t, \quad t \in \{0 \dots N - 1\} \quad (3)$$

$$p_{t+1} = \text{LSTM}(x_t), \quad t \in \{0 \dots N - 1\} \quad (4)$$

With this, our loss function would be the sum of the negative log likelihood of the correct word at each time step t :

$$L(I, S) = - \sum_{t=1}^N \log p_t(S_t) . \quad (5)$$

Finally, for the evaluation part, the input for $t+1$ would be the embedding of the word with the maximum likelihood at t . This can be represented as follows:

$$x_t = \text{argmax}(p_t(S_t)), \quad t \in \{0 \dots N - 1\} \quad (6)$$

2 System Implemented

2.1 Data Downloading and Processing

The data used is the Flickr8K dataset from Kaggle and the code obtains images and captions from the web. Images are divided into train, validation and test folders and resize into (299, 299) dimensions since is needed for the pretrained model. The “captions.txt” file is then transformed into 3 different jsons, each containing the name and 5 captions of each image. To process the captions, we have implemented the “create_lookup_tables” function return the word to index and index to word dictionary. These functions are under the assumption that all the possible vocabulary is contained in the train captions and include the “<PAD>” token. The data in the appropriate format is saved into objects of type “ImageAndCaptionsDataset” to be fed to the Dataloaders.

2.2 Model and Training

The model has an encoder-decoder infrastructure. The encoder is based on an “inception” version 3 pretrained model. For the implementation, we have developed a class “ModifiedInception”, that allows to finetune the last linear layer of the model and applies a relu and dropout to avoid overfitting. The output of this model will be a tensor with the dimension of the word embeddings that will have the main features encoded. This vector will be fed then to the decoder model, which can be found in the “DecoderRNN” class. It must receive the inputs without the batch first. For the forward method, it will concatenate the features vector with the caption provided and fed it through an LSTM torch model. Afterwards, the outputs will go through a linear layer. This encoder-decoder infrastructure is unified in the “ImageCaptioningModel” class, that also has a method “generate_caption”. To test the model, main, train and evaluate functions have been implemented. They use other additional functions such as “train_step” that can be found in the “utils” file. By executing the main file a new model will be trained and in each epoch the progress made will be saved. If training is stopped, it can be resumed by turning the “need_load” variable “True” in the train file. For the evaluation process, the last checkpoint will be loaded and it will use the “generate_caption” method of the model to create descriptions for the test images. For each image, a new one containing the original one, as well as the real and predicted caption will be generated into the “solutions” folder.

3 Preliminary Results

By now we have already trained a model with 30 epochs and have obtained some results. First of all, with few epochs (20) we can see that the model starts differentiating between a man and a dog, however, it predicts similar phrases in all of the test dataset. Our hope is that with more epochs the model will improve.



Figure 1: Captions Obtained

4 Planning for Next Sessions

As it has been explained, we have succeeded with the objectives proposed for the first week since the data has already been processed and we are currently training models. For this new week, we plan to finish fixing bugs in the code and try different parameters for the model. In addition, error metrics have already been implemented but haven't been tried so our next task will be to add them into the training and evaluation loops to compare models. Finally, since we have developed code for different models (vgg19 and inception v3) and datasets (Flick8k and MSCOCO), we plan to clean the code and create a final stable version of it.

References

- [1] Oriol Vinyals et al. “Show and tell: A neural image caption generator”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2015, pp. 3156–3164.